

```
=====
Spring Core module
=====
```

=> Base module of Spring Framework.

=> Providing fundamental concepts of spring framework

- 1) IOC
- 2) Dependency Injection
- 3) Auto wiring

Spring Core module is used to manage our classes in the project.

=> In a project we will have several classes

- 1) Controller classes (handle request & response)
- 2) Service classes (handle business logic)
- 3) DAO classes (handle DB Operations)

=> In project execution process, One java class should call another java class method

- 1) Controller class method should call service class method
- 2) Service class method should call DAO class method

=> We have 2 options to access one java class method in another java class

- 1) Inheritance (IS-A)
- 2) Composition (HAS-A)

```
=====
IS-A Relation
=====
```

=> Extending the properties from one class to another class.

=> Super class methods we can access directly in sub class.

Ex: Car & Engine

Car class ----> drive () method

Engine class ---> start () method

Note: If we want to drive the car then we need to start the Engine first. That means Car class functionality is depending on Engine class functionality.

=> Car class drive () method should call Engine class start () method.

```
-----
```

```
package in.ashokit;
```

```
public class Engine {
```

```
    public boolean start() {
        // logic
        return true;
    }
```

```
}
```

```
-----
```

```
package in.ashokit;
```

```

public class Car extends Engine {

    public void drive() {

        boolean isStarted = start();

        if (isStarted) {
            System.out.println("Journey Started..");
        } else {
            System.out.println("Engine Trouble...");
        }
    }
}

```

=> In the above approach car is extending properties from Engine class.

=> In future car can't extend props from other classes bcz java doesn't support multiple inheritance.

=> With IS-A relationship our classes will become tightly coupled.

=> To overcome problems of IS-A relation we can use HAS-A relation.

=====

HAS-A relation

=====

=> Create object and call the method

=> Inside car class create object for Engine class and call eng class method.

```

public class Car {

    Engine eng = new Engine();

    public void drive() {

        boolean status = eng.start();
        if (status) {
            System.out.println("Journey started...");
        } else {
            System.out.println("Engine having trouble...");
        }
    }
}

```

=> If someone modify Engine class constructor then Car class will get effected...

=> with HAS-A relation also our java classes becoming tightly coupled.

=> To make our classes loosely coupled, we should not extend properties and we should not create object directly.

=> To make our classes loosely coupled we can use Spring Core Module concepts

1) IOC Container

2) Dependency Injection

=====

What is IOC Container

=====

=> IOC stands for Inversion of control.

=> IOC is used to manage & collaborate the classes and objects available in the application.

=> IOC will perform Dependency Injection in our application.

=> Injecting Dependent class object into target class object is called as Dependency Injection.

=> By using IOC and DI we can achieve Loosely coupling among the classes in our application.

Note: We need to provide input for IOC regarding our target classes and dependent classes to perform Dependency Injection.

Note: We can do configuration in 3 ways

1) XML (outdated -> springboot will not support)

2) JAVA Based

3) Annotations (Trending)

=> IOC will take our normal java classes as input and it provides Spring Beans as output.

=====
What is Spring Bean
=====

=> The java class which is managed by IOC is called as Spring bean.

=====
First App development using Spring IOC
=====

Step-1 : Create maven project using IDE (Eclipse/ STS / IntelliJ)

- select simple project (standalone)
- groupId : in.ashokit
- artifactId : 01-Spring-App

Step-2 : Configure Spring dependency in project pom.xml file to download required libraries.

@@@ URL : <https://mvnrepository.com/>

```
-----
<dependencies>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>6.2.10</version>
  </dependency>
</dependencies>
-----
```

Step-3 :: Create Required java classes

```
-----
public class Engine {

    public Engine() {
        System.out.println("Engine :: Constructor");
    }

    public boolean start() {
        System.out.println("Engine starting...");
        return true;
    }

}
```

Step-4 :: Create Spring Bean Configuration file and configure java classes as spring beans.

File Location : src/main/resources/spring-beans.xml

```
-----
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
      xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="eng" class="in.ashokit.beans.Engine"/>

</beans>
-----
```

Step-5 :: Create Main class to test our application.

```
-----
public class App {

    public static void main(String[] args) {

        // Start IOC Container
        ApplicationContext ctxt = new ClassPathXmlApplicationContext("spring-beans.xml");

        // Get Bean obj from IOC
        Engine engObj = ctxt.getBean(Engine.class);

        // Calling bean method
        engObj.start();

    }
}
-----
```

=====

What is Dependency Injection

=====

=> The process of injecting one class object into another class object is called as dependency injection.

Note: IOC is responsible to perform dependency injection.

=> We can perform Dependency Injection in 3 ways

- 1) Construtor Injection
- 2) Setter Injection
- 3) Field Injection

=====

What is Constructor Injection ?

=====

=> Injecting dependent obj into target obj using target class parameterized constructor is called Constructor injection (C.I).

```
// param constructor with dependent obj as constructor arg.
public Car(Engine eng) {
    this.eng = eng;
}
```

Note: To represent constructor injection we will use below syntax

Syntax : <constructor-arg name="" ref=""/>

Ex :

```
<bean id="c" class="in.ashokit.Car">
    <constructor-arg name="eng" ref="e1"/>
</bean>
```

```
<bean id="e1" class="in.ashokit.Engine" />
```

=====

What is Setter Injection ?

=====

=> Injecting dependent obj into target obj using target class setter method is called as setter injection (S.I).

```
// SETTER METHOD with dependent obj as parameter
public void setEng(Engine eng) {
    this.eng = eng;
}
```

Note: To represent setter injection we will use below syntax

Syntax : <property name="" ref=""/>

Ex :

```
<bean id="c" class="in.ashokit.Car">
    <property name="eng" ref="e"/>
</bean>
```

```
<bean id="e" class="in.ashokit.Engine" />
```

=====

Q) For single variable if we perform both CI and SI then what will happen ?

=====

```
<bean id="e1" class="in.ashokit.beans.Engine" />
```

```
<bean id="car" class="in.ashokit.beans.Car">
    <constructor-arg name="eng" ref="e1" />
    <property name="eng" ref="e1" />
</bean>
```

Note: To call setter method, first object should be created. When object is created constructor will get executed then CI will happen. After constructor execution setter method will be called and it will perform setter injection, hence SI will override CI value.

=====

IOC with DI Example

=====

Requirement : When we withdraw amount from ATM then it should print receipt using Printer.

Step-1 : Create Maven project and add 'spring-context' dependency

Step-2 : Create Iprinter interface & its impl classes

Step-3 : Create ATM class

Step-4 : Configure classes as spring beans in beans configuration xml file

Step-5 : Create main class to test the application.

=====

IOC with DI Usecase

=====

Requirement : Develop Spring IOC application with User Registration Functionality

When user register in the application then we should generate random pwd for user and send that pwd to user in email.

1) UserService.java

2) PwdService.java

3) EmailService.java

Note: UserService class should communicate with PwdService and EmailService classes.

=====
Bean Scopes
=====

=> Bean scope represents how many objects should be created for spring bean by IOC container.

=> We have below bean scopes in spring

1) Singleton (default)

2) Prototype

3) Request

4) Session

Singleton

=> singleton is default scope.

=> Only one instance will be created for spring bean.

=> Singleton scoped bean objects will be created when IOC container started.

=> Singleton beans will follow Eager Loading.

Note: Even if we call getBean() method multiple times, IOC will return same object.

Prototype

=> Every time new object will be created for spring bean on demand basis.

=> Prototype beans will follow lazy loading.

=> When we call getBean() method then only obj will be created.

request & session

=> These 2 scopes are belongs to spring web mvc module.

=====

1) Core JAVA vs Adv Java

2) What is Framework

3) JAVA Frameworks

- 4) Struts Vs Hibernate Vs Spring
- 5) Spring Introduction
- 6) Spring Architecture
- 7) Spring Modules Overview
- 8) Spring Core Module
- 9) IOC Container
- 10) Dependency Injection
- 11) Constructor Injection
- 12) Setter Injection
- 13) Bean Scopes